Next.js Mastery Series — Topic 10 of N

Performance Optimization

Core Web Vitals · Images · Fonts · Dynamic Imports · Caching · Bundle Analysis

Akshay Chavhan | Target: Syngenta India + All Companies | April 2026

■ BEGINNER LEVEL

Core Web Vitals — Google's Performance Standard

Core Web Vitals are Google's three key metrics for measuring user experience. Poor scores hurt SEO ranking and increase bounce rate.

LCP
Largest Contentful Paint
Main content load speed
Goal: < 2.5s

INP
Interaction to Next Paint
Response to clicks/taps
Goal: < 200ms

CLS
Cumulative Layout Shift
Visual stability
Goal: < 0.1

1. Image Optimization — next/image

The **biggest and easiest performance win** in Next.js. Replace every regular img tag with the Image component.

```
// ■ Regular img — NO optimization
<img src="/crop.jpg" alt="Wheat" />
// Loads full 5MB image on mobile, no lazy load, no WebP

// ■ next/image — everything automatic
import Image from 'next/image';

// Basic usage
<Image src="/crop.jpg" alt="Wheat" width={800} height={600} />

// Hero image (above fold) — add priority to preload
<Image src="/hero.jpg" alt="Banner" width={1200} height={600}
  priority  // preloads = faster LCP
/>

// Responsive — serves correct size per device
<Image src="/product.jpg" alt="Seeds" width={400} height={300}
  sizes="(max-width: 768px) 100vw, (max-width: 1200px) 50vw, 33vw"
/>
// Mobile: full width | Tablet: half | Desktop: one-third
```

Feature	Regular img	next/image
Auto resize per device	No	Yes
Lazy loading	No	Yes (default)
WebP/AVIF format	No	Yes (auto-convert)

Prevents layout shift	No	Yes (reserves space)
Blur placeholder	No	Yes (blurDataURL)
Priority preload	No	Yes (priority prop)

2. Font Optimization — next/font

```
// ■ Old way — Google Fonts via link tag
// <link href="https://fonts.googleapis.com/css2?family=Inter" />
// Problems: extra DNS lookup, blocks render, layout shift

// ■ next/font — self-hosted, zero layout shift
import { Inter } from 'next/font/google';

const inter = Inter({
  subsets: ['latin'],
  display: 'swap',   // show fallback first, swap when ready
  weight: ['400', '600', '700'],  // only what you need
});

export default function RootLayout({ children }) {
  return (
    <html lang="en" className={inter.className}>
      <body>{children}</body>
    </html>
  );
}
// Font downloaded at BUILD TIME — no runtime network request ■
// No DNS lookup to Google — your server hosts it ■
// Zero layout shift — size-adjust CSS applied automatically ■
```

■ INTERMEDIATE LEVEL

3. Dynamic Imports — Code Splitting

By default Next.js includes ALL component code in the initial bundle. Dynamic imports load code **only when the component actually renders**.

```
import dynamic from 'next/dynamic';

// ■ Normal import — 200KB chart library always in bundle
import HeavyChart from '@/components/Chart';

// ■ Dynamic — only loads when component renders
const Chart = dynamic(() => import('@/components/Chart'), {
  loading: () => <p>Loading chart...</p>,
  ssr: false,  // chart uses browser APIs — don't render on server
});

// Conditional loading — admin panel only downloads when opened
const AdminPanel = dynamic(() => import('./AdminPanel'));
'use client';
import { useState } from 'react';
function Dashboard() {
  const [show, setShow] = useState(false);
  return (
    <div>
      <button onClick={() => setShow(true)}>Open Admin</button>
      {show && <AdminPanel />}  {/* JS downloads HERE only */}
    </div>
  );
}
```

Use ssr: false when...	Example
Component uses window/document	Map, video player
Component uses localStorage	Cart, theme
Browser-only library	Canvas, WebGL
Chart libraries (DOM-based)	recharts, chart.js

4. Script Optimization — next/script

```
import Script from 'next/script';

// 3 strategies:

// beforeInteractive — loads BEFORE page interactive
// Use for: truly critical scripts only — BLOCKS page!
<Script src="/critical.js" strategy="beforeInteractive" />

// afterInteractive (default) — loads AFTER page interactive
// Use for: analytics, tag managers
<Script
  src="https://www.googletagmanager.com/gtag/js"
  strategy="afterInteractive"
/>

// lazyOnload — loads during browser IDLE time
// Use for: chat widgets, low-priority scripts
<Script
  src="https://widget.chat.com/widget.js"
  strategy="lazyOnload"
/>
```

5. Server Components — Zero JS Bundle

```
// ■ Over-using Client Components — ALL code goes to browser
'use client';
function ProductList({ products }) {
  const [filter, setFilter] = useState("all");
  // Entire list code ships to browser unnecessarily
}

// ■ Server Component for display, Client only for interaction
// ProductList.tsx — Server Component (default)
async function ProductList() {
  const products = await db.product.findMany(); // server only
  return (
    <div>
      <FilterBar />            {/* Only this is Client */}
      {products.map(p => (
        <ProductCard key={p.id} product={p} /> // Server rendered
      ))}
    </div>
  );
}

// FilterBar.tsx — Client Component (leaf, smallest possible)
'use client';
function FilterBar() {
  const [filter, setFilter] = useState("all");
  return <select onChange={e => setFilter(e.target.value)}>...</select>;
}
// Result: 90% of page = 0 JS to browser ■
```

6. Suspense + Streaming — Progressive Loading

```
import { Suspense } from 'react';

export default function DashboardPage() {
  return (
    <div>
      <h1>Syngenta Dashboard</h1>  {/* Instant — no data */}

      {/* Streams in when crop DB query completes */}
      <Suspense fallback={<p>Loading prices...</p>}>
        <CropPricesTable />
      </Suspense>

      {/* Independent — does NOT wait for CropPricesTable */}
      <Suspense fallback={<p>Loading chart...</p>}>
        <PriceHistoryChart />
      </Suspense>
    </div>
  );
}
// Without Suspense: blank page until ALL data loads
// With Suspense:    content streams in progressively ■
```

■ EXPERT LEVEL

7. Bundle Analyzer — Find Large Dependencies

```
npm install @next/bundle-analyzer

// next.config.js
const withBundleAnalyzer = require('@next/bundle-analyzer')({
  enabled: process.env.ANALYZE === "true",
});
module.exports = withBundleAnalyzer({});

// Run: ANALYZE=true npm run build
// Opens visual map — find large packages

// Common fixes after analyzing:

// ■ Import entire lodash (70KB)
import _ from 'lodash';

// ■ Import only what you need (2KB)
import debounce from 'lodash/debounce';

// ■ moment.js (67KB)
import moment from 'moment';

// ■ date-fns (2KB per function)
import { format } from 'date-fns';

// ■ All react-icons in bundle
import { FaHome } from 'react-icons/fa'; // loads ALL icons!

// ■ Dynamic import specific icon
const FaHome = dynamic(() =>
  import('react-icons/fa').then(m => m.FaHome)
);
```

8. Parallel Data Fetching — No Waterfalls

```
// ■ Sequential — 750ms total (waterfall!)
async function SlowPage() {
  const user    = await getUser();    // 200ms — waits
  const crops   = await getCrops();   // 300ms — waits after user
  const weather = await getWeather(); // 250ms — waits after crops
  // Total: 750ms ■
}

// ■ Parallel — 300ms total
async function FastPage() {
  const [user, crops, weather] = await Promise.all([
    getUser(),    // 200ms ■■
    getCrops(),   // 300ms  ■■■ all start at same time
    getWeather(), // 250ms ■■
  ]);
  // Total: 300ms (only slowest matters) ■
}
```

9. React.cache — Request-Level Deduplication

```
import { cache } from 'react';
import prisma    from '@/lib/prisma';

// cache() = deduplicate within ONE request
export const getUser = cache(async (userId: string) => {
  console.log("DB hit:", userId); // Only prints ONCE per request
  return prisma.user.findUnique({ where: { id: userId } });
});

// layout.tsx calls getUser("123")  → DB query fires
// page.tsx   calls getUser("123")  → Returns cached result ■
// Result: only 1 DB query instead of 2

// DIFFERENCE from unstable_cache:
// React.cache     = per-request, resets between requests
// unstable_cache  = persists across requests (all users share)
```

10. Performance Checklist

Area	Optimization	Impact
Images	next/image everywhere + priority on hero	LCP improvement
Images	Add sizes prop for responsive images	Smaller files on mobile
Fonts	next/font instead of link tags	CLS fix + faster load
Bundle	dynamic() for heavy components	Smaller initial JS
Bundle	Named imports not whole libraries	Smaller bundle
Server/Client	Push use client to leaf components	Less JS to browser
Data	Promise.all for independent fetches	Faster page data
Data	unstable_cache for Prisma queries	Fewer DB hits
Rendering	Suspense boundaries around slow sections	Progressive load
Scripts	afterInteractive/lazyOnload strategy	No render blocking
Analysis	Run bundle analyzer regularly	Find large deps early

■■ Common Performance Mistakes

■■ Mistake 1: Using regular img tag instead of next/image. This is the single biggest LCP killer. Every img tag = no optimization, no lazy loading, no WebP, potential layout shift.

■■ Mistake 2: Putting 'use client' at the top of page.tsx or layout.tsx. This makes the ENTIRE page a Client Component — all children become Client Components, entire page JS ships to browser.

■■ Mistake 3: Sequential awaits for independent data fetches. Three sequential fetches of 200ms + 300ms + 250ms = 750ms. Promise.all makes them run in parallel = 300ms. Always use Promise.all for independent data.

■■ Mistake 4: Importing entire libraries. import _ from 'lodash' = 70KB. import { debounce } from 'lodash/debounce' = 2KB. Run bundle analyzer to find these.

■ Best Practice: Add priority prop to the first/hero image on every page. This is the fastest LCP win — costs zero effort, gains 200-500ms on LCP score.

■ Best Practice: Wrap every slow data-fetching section in Suspense. Users see content progressively instead of a blank page. Works automatically with loading.tsx too.

■ INTERVIEW Q&A WITH DETAILED ANSWERS

Beginner Level

BEGINNER

Q1. What are Core Web Vitals and why do they matter?

Answer: Three Google metrics: LCP (Largest Contentful Paint — main content load, target under 2.5s), INP (Interaction to Next Paint — click response time, target under 200ms), CLS (Cumulative Layout Shift — visual stability, target under 0.1). Google uses these for SEO ranking. Poor scores = lower search position + higher bounce rate.

BEGINNER

Q2. What does next/image do that a regular img tag does not?

Answer: Automatic resizing per device (mobile gets small, desktop gets large), lazy loading by default (below-viewport images load only when scrolled to), automatic format conversion to WebP/AVIF (30-50% smaller files), reserves space preventing layout shift. Add priority prop to hero/above-fold images for faster LCP.

BEGINNER

Q3. Why use next/font instead of Google Fonts link tag?

Answer: Link tag makes a runtime request to Google servers (extra DNS lookup, blocks render, layout shift). next/font downloads the font at build time and self-hosts it on your server. No DNS lookup, no blocking, zero layout shift (size-adjust CSS automatic). Also works offline and is faster for users everywhere.

Intermediate Level

INTERMEDIATE

Q4. What is dynamic import and when do you use it?

Answer: dynamic() loads a component's JavaScript only when it renders — not upfront. Use for: heavy chart/editor libraries to reduce initial bundle, browser-only components with ssr:false (map, video, canvas), and conditionally shown UI like modals and admin panels. Result: smaller initial JS bundle = faster first load = better LCP and INP.

INTERMEDIATE

Q5. What is the difference between ssr:false in dynamic() and a regular Client Component?

Answer: A Client Component with use client still gets server-rendered to HTML on first load for SEO. dynamic(() => import(...), { ssr: false }) completely skips server rendering — the component only renders in the browser. Use ssr:false for components using window, document, or localStorage which would crash on server.

INTERMEDIATE

Q6. How do you avoid data fetching waterfalls in Next.js?

Answer: Use Promise.all() for independent fetches instead of sequential awaits. Sequential: 200ms + 300ms + 250ms = 750ms total. Parallel with Promise.all: 300ms total (only slowest). Also use Suspense to split slow sections so fast data shows while slow data loads. This is one of the easiest and biggest performance wins.

INTERMEDIATE

Q7. How does Suspense improve performance?

Answer: Without Suspense: page waits for ALL data before any HTML is sent. With Suspense: sections stream independently. Fast sections show immediately. Slow sections (wrapped in Suspense) trickle in as their data resolves. Users see content progressively instead of one long blank wait. loading.tsx is an automatic Suspense boundary per route segment.

Expert / Syngenta-Style Questions

EXPERT

Q8. How do you reduce JavaScript bundle size in Next.js?

Answer: Run @next/bundle-analyzer to visualize what is large. Then: use named imports not whole libraries (lodash/debounce not lodash), replace heavy libraries with lighter ones (date-fns instead of moment), dynamically import heavy components with dynamic(), push use client to leaf components only (Server Components add zero JS), remove unused dependencies with npm prune.

EXPERT

Q9. What is React.cache() and how is it different from unstable_cache?

Answer: React.cache() deduplicates function calls within ONE server request. If layout and page both call getUser(id), only one DB query fires. Resets between requests. unstable_cache persists results across requests and users — it is the server-side ISR cache. Use React.cache for per-request deduplication. Use unstable_cache for persistent cross-request caching with revalidation.

SYNGENTA

Q10. A Syngenta crop dashboard is loading slowly. How do you diagnose and fix it?

Answer: Step 1: Run Lighthouse — identify LCP, CLS, INP scores. Step 2: Run bundle analyzer — find large imports. Step 3: Add next/image to all images with priority on hero. Step 4: Replace Google Fonts link with next/font. Step 5: Use Promise.all for parallel data fetching + unstable_cache for Prisma. Step 6: Push use client to leaf components only, wrap chart in dynamic() with ssr:false. Step 7: Add Suspense around slow sections so fast sections show immediately.

EXPERT

Q11. What is the impact of putting use client on a top-level layout?

Answer: Catastrophic for performance. Every component in the entire subtree becomes a Client Component. The entire React tree gets bundled into JavaScript sent to the browser — defeating all Server Component optimizations. No component can benefit from zero-JS server rendering anymore. Rule: layouts should be Server Components. Only add use client to the smallest interactive leaf components.